

FedPrivacyLab

Federated Analytics and Privacy-Preserving ML Workbench

Graduate-Level Architecture Specification and Cursor Build Guide

"Built a federated analytics and learning workbench in Python that simulates decentralized clients, runs aggregate analytics and federated model training without centralizing raw data, implements FedAvg, client selection, dropout simulation, update clipping, differential privacy noise, secure aggregation simulation, and dashboards for privacy-utility tradeoff analysis."

Primary Goal	Interview Value
Federated analytics	Compute useful aggregate metrics without collecting raw client records.
Federated learning	Show local training, server aggregation, FedAvg, and client heterogeneity.
Privacy engineering	Show FL alone is not enough; add bounding, clipping, DP, secure aggregation, and aggregate-only reports.
Systems engineering	Build coordinator APIs, client simulators, dashboards, tests, and reproducible experiments.

Prepared for Apple AIML Privacy Engineering Rotation HM Interview

Focus: Privacy-preserving measurement, federated analytics, federated learning, and defensible project storytelling.

1. Project Positioning

"I built FedPrivacyLab to understand privacy-preserving analytics and federated learning end to end. The system simulates clients with local data, runs analytics or model training locally, sends only bounded updates or aggregate reports, and compares plain federated aggregation, secure aggregation simulation, and DP-noised aggregation. The main learning was that federated systems reduce raw-data centralization, but privacy still needs to be designed carefully."

This project should not be described as only a federated learning notebook. It should be described as a federated analytics plus federated learning privacy workbench. That framing is stronger for the Apple role because the team works across privacy-preserving measurement and ML systems.

2. Final Product Scope

Mode 1: Federated Analytics

This mode is closest to privacy-preserving measurement. The system answers aggregate product questions without collecting raw logs from clients.

Questions supported:

- How many clients saw a feature failure?
- What was the average latency by cohort?
- What percentage of clients clicked a recommendation?
- Which feature had the highest crash rate?

Each client computes a local summary instead of sending raw event logs:

```
{
  "feature": "writing_tools",
  "failure_count": 3,
  "success_count": 12,
  "latency_sum": 8240,
  "latency_count": 12
}
```

- Contribution bounds
- Secure aggregation simulation
- Differential privacy noise
- Cohort thresholds

Mode 2: Federated Learning

This mode trains a global model from decentralized client data without uploading raw client records. Each client trains locally, sends bounded model updates, and the server performs FedAvg.

Core question:

Can we train a global model from decentralized client data without uploading raw records?

3. Recommended Tech Stack

Layer	Recommended Tools
Backend API	FastAPI, Pydantic, SQLAlchemy
Database	SQLite for MVP; PostgreSQL optional
Data and ML	NumPy, Pandas, scikit-learn, TensorFlow/Keras
Dashboard	Streamlit

Configuration	PyYAML
Testing	pytest
Optional FL framework	Flower or TensorFlow Federated after MVP

Recommended implementation strategy: build custom NumPy/TensorFlow orchestration first so the FedAvg, clipping, secure aggregation simulation, and DP logic are visible and defensible. Optional Flower or TensorFlow Federated integration can be added later.

4. Dataset Strategy

Dataset A: Synthetic Client Telemetry Dataset

This is the primary dataset because it directly connects to privacy-preserving measurement systems.

- Generate 5,000 to 50,000 synthetic clients.
- Each client stores local telemetry records.
- Raw records stay local to simulated clients.
- Use data for both federated analytics and federated binary classification.

```
{
  "client_id": "client_00042",
  "locale": "en_US",
  "app_version": "1.3",
  "device_tier": "mid",
  "network_type": "wifi",
  "feature": "writing_tools",
  "latency_ms": 950,
  "failure": 0,
  "clicked_recommendation": 1,
  "session_length_sec": 180,
  "label_poor_experience": 0
}
```

Field	Use
latency_ms	Performance signal and model feature
failure_count / failure	Feature reliability signal
clicked_recommendation	Engagement signal
session_length_sec	User experience signal
device_tier, network_type, app_version, locale_bucket	Cohort and non-IID simulation features
label_poor_experience	Binary learning target

Dataset B: Federated EMNIST

Optional benchmark dataset. Use it only after the synthetic telemetry flow works. It can show awareness of standard federated learning benchmarks, but it should not distract from the role-aligned telemetry project.

Dataset C: LEAF Benchmark

Optional extension. Mention compatibility or future support for LEAF/FEMNIST or Shakespeare, but do not overbuild it before the interview.

5. High-Level Architecture

```
Experiment Config
|
v
```

Client Data Generator / Loader

|

+--> Synthetic Telemetry Clients

+--> Federated EMNIST Clients (optional)

+--> LEAF Clients (optional)

|

v

Client Registry

|

v

Round Coordinator

|

+--> Federated Analytics Engine --> Local Client Summaries

|

+--> Federated Learning Engine --> Local Model Training

|

v

Contribution Bounding / Update Clipping

|

v

Secure Aggregation Simulator

|

v

Differential Privacy Engine

|

v

Server Aggregator

|

+--> Global Analytics Metrics

+--> Global Model

|

v

Privacy and Utility Evaluator

|

v

Streamlit Dashboard

6. End-to-End System Flow

1. Experiment runner starts a configuration.
2. Coordinator loads client population.
3. Coordinator selects clients for the current round.
4. Selected clients receive an analytics query or global model.
5. In analytics mode, clients compute local summaries.
6. In learning mode, clients train locally and compute model updates.
7. Privacy layer bounds contributions or clips updates.
8. Secure aggregation simulator masks individual summaries/updates.
9. Server aggregates only the combined result.
10. Differential privacy noise is optionally added.
11. Metrics, privacy reports, and utility comparisons are saved.
12. Dashboard visualizes privacy-utility tradeoffs.

7. Main Modules and Folder Structure

```
fedprivacylab/  
  README.md  
  requirements.txt  
  docker-compose.yml  
  
  config/  
    experiment_config.yaml  
    privacy_config.yaml  
  
  app/  
    main.py  
    database.py  
    models.py  
    schemas.py  
  
  api/  
    experiment_routes.py  
    analytics_routes.py  
    learning_routes.py  
    privacy_routes.py  
    metrics_routes.py  
  
  coordinator/  
    round_coordinator.py  
    client_registry.py  
    experiment_runner.py  
  
  clients/  
    client.py  
    telemetry_client.py  
    emnist_client.py  
    local_analytics.py  
    local_training.py  
  
  analytics/  
    federated_analytics.py  
    local_summary.py  
    histogram.py  
    quantiles.py  
  
  learning/  
    model.py  
    fedavg.py  
    centralized_baseline.py  
    evaluator.py  
  
  privacy/  
    contribution_bounding.py  
    update_clipping.py  
    secure_aggregation.py  
    dp_mechanisms.py  
    privacy_accountant.py  
    risk_report.py
```

```

simulation/
  synthetic_telemetry.py
  non_iid_partition.py
  dropout_simulator.py
  attack_simulator.py

dashboard/
  streamlit_app.py
pages/
  overview.py
  federated_analytics.py
  training_progress.py
  privacy_controls.py
  utility_tradeoff.py
  client_participation.py
  risk_report.py

tests/
  test_fedavg.py
  test_contribution_bounding.py
  test_update_clipping.py
  test_secure_aggregation.py
  test_dp_mechanisms.py
  test_non_iid_partition.py

```

8. Federated Analytics Mode

Federated analytics is the most Apple-role-specific part because it mirrors privacy-preserving measurement: clients compute local summaries, not raw logs.

Analytics Query	Meaning
count_failures_by_feature	Total feature failures across selected clients
average_latency_by_feature	Average latency from clipped local sums
click_rate_by_feature	Recommendation click rate from local counts
histogram_latency_by_bucket	Latency distribution using bounded buckets
daily_active_clients_by_cohort	Cohort-level active client count
poor_experience_rate	Aggregate rate of poor experience labels

Client raw local records:

```
[
  {"feature": "writing_tools", "latency_ms": 1200, "failure": 1},
  {"feature": "writing_tools", "latency_ms": 850, "failure": 0},
  {"feature": "visual_search", "latency_ms": 1500, "failure": 1}
]
```

Client sends bounded summary only:

```
{
  "client_id_removed_before_storage": true,
  "round_id": 1,
  "feature": "writing_tools",
  "failure_count": 1,
  "event_count": 2,
  "latency_sum_clipped": 2050,
}
```

```
"latency_count": 2
}
```

Global computations:

```
global_failure_rate = total_failure_count / total_event_count
global_avg_latency = total_latency_sum / total_latency_count
```

Contribution Bounding for Analytics

```
max_events_per_client_per_round: 20
max_failures_per_client_per_round: 5
max_latency_ms_per_event: 5000
max_latency_sum_per_client: 50000
```

"Before adding DP noise, I bound each client contribution so one client cannot dominate the aggregate."

9. Federated Learning Mode

The learning task is a binary classifier that predicts poor user experience from telemetry features.

Prediction target:

```
label_poor_experience = 1 if high latency, repeated failure, or low engagement
label_poor_experience = 0 otherwise
```

Model Item	Specification
Input features	latency_ms, failure_count, click_count, session_length_sec, device_tier, network_type, app_version, locale_bucket
Architecture	Dense(32, relu) -> Dense(16, relu) -> Dense(1, sigmoid)
Loss	binary_crossentropy
Metrics	accuracy, ROC-AUC, precision, recall, F1

FedAvg Flow

13. Server sends current global model weights.
14. Coordinator selects K clients.
15. Each selected client trains locally for E epochs.
16. Client computes weight delta.
17. Client update is clipped by L2 norm.
18. Secure aggregation simulator aggregates updates.
19. Differential privacy noise is added if enabled.
20. Server applies weighted average update.
21. Global model is evaluated.

FedAvg formula:

```
global_update = sum((n_i / N) * client_update_i)
```

where:

```
n_i = examples on client i
N = total examples across selected clients
```

"I implemented FedAvg manually so I could understand the mechanics instead of relying only on a framework. Each client trains locally, sends a bounded model update, and the server averages updates weighted by local data size."

10. Privacy Layer

10.1 Contribution Bounding

```
bounded_failure_count = min(failure_count, max_failures_per_client)
bounded_latency_sum = min(latency_sum, max_latency_sum_per_client)
```

10.2 Update Clipping

```
def clip_update(update_vector, max_norm):
    norm = np.linalg.norm(update_vector)
    if norm <= max_norm:
        return update_vector
    return update_vector * (max_norm / norm)
```

"Clipping bounds how much one client can influence the global model update."

10.3 Differential Privacy Noise

For analytics, use Laplace noise for counts and Gaussian noise for bounded sums/averages. For learning, add Gaussian noise after update clipping.

```
Analytics count:
noisy_count = count + Laplace(0, sensitivity / epsilon)

Model update:
noisy_aggregate_update = clipped_aggregate_update + Normal(0, noise_multiplier * clipping_norm)
```

10.4 Secure Aggregation Simulation

```
client_update_masked = client_update + random_mask
aggregator sums masked updates
masks cancel out in aggregate
server sees aggregate only
```

"In my prototype, secure aggregation is simulated to demonstrate the privacy design. I am not claiming it is a production cryptographic implementation."

10.5 Dropout Simulation

- Simulate 10%, 20%, and 30% client dropout rates.
- Track selected clients, completed clients, dropped clients, and impact on accuracy.
- Use this to show that real federated systems face system heterogeneity.

11. Experiment Modes

Mode	Raw Data Centralized?	Individual Update Visible?	DP Noise?	Utility
centralized_baseline	Yes	N/A	No	Highest
federated_analytics	No	Local summaries may	Optional	High

		be visible		
fedavg	No	Yes	No	High
fedavg_secureagg	No	No	No	High
fedavg_secureagg_dp	No	No	Yes	Lower but safer

12. Database Schema

```
CREATE TABLE experiments (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  mode TEXT NOT NULL,
  dataset TEXT NOT NULL,
  num_clients INTEGER,
  rounds INTEGER,
  clients_per_round INTEGER,
  local_epochs INTEGER,
  dp_enabled BOOLEAN,
  secure_agg_enabled BOOLEAN,
  dropout_rate REAL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE round_metrics (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  experiment_id INTEGER,
  round_number INTEGER,
  selected_clients INTEGER,
  completed_clients INTEGER,
  dropped_clients INTEGER,
  train_loss REAL,
  eval_loss REAL,
  accuracy REAL,
  roc_auc REAL,
  precision_score REAL,
  recall_score REAL,
  f1_score REAL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE analytics_results (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  experiment_id INTEGER,
  round_number INTEGER,
  metric_name TEXT,
  feature TEXT,
  true_value REAL,
  federated_value REAL,
  dp_noisy_value REAL,
  epsilon REAL,
  absolute_error REAL,
  relative_error REAL,
  suppressed BOOLEAN,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
);
```

```
CREATE TABLE privacy_reports (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  experiment_id INTEGER,  
  round_number INTEGER,  
  privacy_mode TEXT,  
  clipping_norm REAL,  
  noise_multiplier REAL,  
  epsilon REAL,  
  delta REAL,  
  secure_agg_enabled BOOLEAN,  
  contribution_bound INTEGER,  
  risk_level TEXT,  
  notes TEXT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE client_participation (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  experiment_id INTEGER,  
  round_number INTEGER,  
  client_id_hash TEXT,  
  selected BOOLEAN,  
  completed BOOLEAN,  
  num_examples INTEGER,  
  update_norm REAL,  
  clipped BOOLEAN,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Use hashed synthetic client IDs only for experiment debugging. In a production privacy-preserving system, persistent client identifiers should be avoided or handled under stricter privacy constraints.

13. API Endpoints

```
POST /api/v1/experiments/start  
POST /api/v1/experiments/{id}/run-round  
GET /api/v1/experiments/{id}/metrics  
GET /api/v1/experiments/{id}/privacy-report  
GET /api/v1/experiments/{id}/analytics  
GET /api/v1/experiments/{id}/client-participation  
POST /api/v1/simulate/clients  
POST /api/v1/simulate/non-iid  
GET /api/v1/health
```

Start experiment request:

```
{  
  "name": "fedavg_dp_telemetry_experiment",  
  "mode": "fedavg_secureagg_dp",  
  "dataset": "synthetic_telemetry",  
  "num_clients": 500,  
}
```

```

"rounds": 20,
"clients_per_round": 50,
"local_epochs": 2,
"dropout_rate": 0.15,
"secure_agg_enabled": true,
"dp_enabled": true,
"clipping_norm": 1.0,
"noise_multiplier": 0.5,
"epsilon": 2.0,
"delta": 0.000001
}

```

Privacy report response:

```

{
  "experiment_id": 1,
  "privacy_mode": "FedAvg + Secure Aggregation Simulation + DP",
  "raw_data_centralized": false,
  "individual_updates_visible_to_server": false,
  "client_updates_clipped": true,
  "dp_noise_added": true,
  "risk_level": "medium-low",
  "notes": [
    "Raw client data remains local",
    "Secure aggregation is simulated, not production cryptography",
    "DP noise improves privacy but reduces model utility",
    "Client-level privacy accounting is simplified"
  ]
}

```

14. Dashboard Design

Dashboard Tab	What It Shows
Overview	Dataset, clients, rounds, mode, raw data centralized status, DP and secure aggregation status
Federated Analytics	Failure rate, average latency, click rate, true vs federated vs DP-noisy values
Federated Learning Progress	Round vs accuracy, loss, ROC-AUC, precision, recall, F1
Privacy Controls	Secure aggregation, DP, epsilon, noise multiplier, clipping norm, dropout, non-IID severity
Utility Tradeoff	Privacy strength vs accuracy, epsilon vs relative error, noise vs ROC-AUC
Client Participation	Selected, completed, dropped clients, update norms, clipped percentage
Risk Report	Raw data upload risk, update leakage, dropout, poisoning, non-IID risks and mitigations
System Explanation	Plain-English privacy-preserving federated design explanation

System Explanation text:

1. Client data stays local.
2. Server selects clients per round.
3. Clients compute local analytics or train local models.
4. Each client contribution is bounded.
5. Model updates are clipped before aggregation.
6. Secure aggregation is simulated so the server sees only aggregate updates.

- 7. Differential privacy noise can be added to aggregate analytics or model updates.
- 8. Dashboard compares privacy strength with utility loss.

15. Privacy and Risk Model

Threat Model

- The server wants useful aggregate metrics or model updates.
- The server should not see raw client records.
- Individual client updates may reveal information.
- Rare client behavior may leak through aggregates.
- Some clients may drop out.
- Some clients may be malicious or noisy.

Privacy Goals

- No raw client data centralized.
- No raw per-client telemetry logs stored.
- Bounded client contributions.
- Optional secure aggregation simulation.
- Optional DP noise.
- Aggregate-only reporting.
- Suppress small cohorts.

Limitations

- Secure aggregation is simulated, not a production cryptographic protocol.
- Privacy accounting is simplified.
- Synthetic data may not represent real user behavior.
- The project demonstrates design patterns, not Apple-scale production guarantees.

16. Core Implementation Details

16.1 Non-IID Data Generator

```
client_profile = random.choice([
    "fast_wifi",
    "slow_cellular",
    "old_app_version",
    "high_failure",
    "power_user",
    "low_engagement"
])
```

- fast_wifi users have lower latency and fewer failures.
- slow_cellular users have higher latency and more dropout.
- old_app_version users have higher failure probability.
- power_user clients have more events per round.
- low_engagement clients have shorter sessions and fewer clicks.

"I intentionally made the data non-IID because real federated systems do not have perfectly balanced client distributions."

16.2 FedAvg Implementation

```
def fedavg(global_weights, client_updates, client_sizes):
    total_examples = sum(client_sizes)
    weighted_update = 0

    for update, n in zip(client_updates, client_sizes):
        weighted_update += (n / total_examples) * update

    new_global_weights = global_weights + weighted_update
    return new_global_weights
```

16.3 Secure Aggregation Simulation

```
def mask_update(update, seed):
    rng = np.random.default_rng(seed)
    mask = rng.normal(0, 1, size=update.shape)
    return update + mask, mask

def unmask_aggregate(masked_updates, masks):
    return np.sum(masked_updates, axis=0) - np.sum(masks, axis=0)
```

16.4 DP Update Clipping and Noise

```
def clip_update(update, clipping_norm):
    norm = np.linalg.norm(update)
    if norm <= clipping_norm:
        return update, False
    return update * (clipping_norm / norm), True

def add_gaussian_noise(update, clipping_norm, noise_multiplier):
    noise = np.random.normal(
        loc=0.0,
        scale=noise_multiplier * clipping_norm,
        size=update.shape
    )
    return update + noise
```

16.5 Analytics DP

```
def laplace_count(count, sensitivity, epsilon):
    scale = sensitivity / epsilon
    return count + np.random.laplace(0, scale)
```

For averages:

1. Clip each value.
2. Sum clipped values.
3. Add noise to the sum.
4. Divide by bounded count.

17. Cursor Build Prompt

Build a graduate-level federated analytics and federated learning privacy workbench called FedPrivacyLab.

Tech stack:

- Python 3.11+
- FastAPI backend
- SQLite database with SQLAlchemy
- Pydantic schemas
- NumPy, Pandas, scikit-learn
- TensorFlow/Keras for model training
- Streamlit dashboard
- PyYAML config
- pytest tests

Core idea:

The system simulates decentralized clients with local telemetry data. The server should not collect raw client records. The system must support two modes:

1. Federated Analytics: clients compute local summaries such as counts, failure rates, latency sums, and histograms. Server aggregates summaries with contribution bounding, optional secure aggregation simulation, and optional differential privacy noise.
2. Federated Learning: clients train a local model to predict poor user experience from telemetry features. Server performs FedAvg using clipped client updates, optional secure aggregation simulation, and optional DP Gaussian noise.

Dataset:

Primary dataset should be synthetic telemetry:

- Generate 5,000 to 50,000 synthetic clients.
- Each client has local records with fields:
latency_ms, failure, clicked_recommendation, session_length_sec, device_tier, network_type, app_version, locale_bucket, feature.
- Label is label_poor_experience.
- Generate non-IID client profiles:
fast_wifi, slow_cellular, old_app_version, high_failure, power_user, low_engagement.
- Data must remain local to simulated clients.

Optional dataset integration:

- Add stub/loader support for Federated EMNIST later.

Architecture files:

- app/main.py
- app/database.py
- app/models.py
- app/schemas.py
- app/api/experiment_routes.py
- app/api/analytics_routes.py
- app/api/learning_routes.py
- app/api/privacy_routes.py
- app/coordinator/round_coordinator.py
- app/coordinator/client_registry.py
- app/coordinator/experiment_runner.py
- app/clients/client.py
- app/clients/telemetry_client.py
- app/clients/local_analytics.py
- app/clients/local_training.py
- app/analytics/federated_analytics.py
- app/learning/model.py
- app/learning/fedavg.py

- app/learning/centralized_baseline.py
- app/learning/evaluator.py
- app/privacy/contribution_bounding.py
- app/privacy/update_clipping.py
- app/privacy/secure_aggregation.py
- app/privacy/dp_mechanisms.py
- app/privacy/privacy_accountant.py
- app/privacy/risk_report.py
- app/simulation/synthetic_telemetry.py
- app/simulation/non_iid_partition.py
- app/simulation/dropout_simulator.py
- dashboard/streamlit_app.py
- tests for all privacy and aggregation logic.

Experiment modes:

1. centralized_baseline
2. federated_analytics
3. fedavg
4. fedavg_secureagg
5. fedavg_secureagg_dp

Federated analytics requirements:

- Implement local client summaries:
failure_count, event_count, latency_sum, latency_count, click_count, histogram_latency_buckets.
- Apply contribution bounds:
max_events_per_client_per_round, max_failures_per_client_per_round, max_latency_sum_per_client.
- Aggregate local summaries globally.
- Add Laplace noise for count metrics.
- Add Gaussian noise for sum/average metrics.
- Compute true value, federated value, DP-noisy value, absolute error, and relative error.
- Suppress reporting for cohorts with fewer than 100 participating clients.

Federated learning requirements:

- Build a small Keras binary classifier:
Input telemetry features -> Dense(32, relu) -> Dense(16, relu) -> Dense(1, sigmoid)
- Train local client models for local_epochs.
- Compute client weight updates.
- Clip updates by L2 norm.
- Simulate secure aggregation by masking updates and aggregating only the unmasked sum.
- Add Gaussian DP noise to aggregate update if enabled.
- Implement FedAvg weighted by client dataset size.
- Track accuracy, ROC-AUC, precision, recall, F1, loss per round.
- Compare centralized baseline, FedAvg, FedAvg + SecureAgg, FedAvg + SecureAgg + DP.

Privacy features:

- No raw client records should be stored on the server.
- Store only aggregate metrics, round metrics, privacy reports, and synthetic hashed client participation for simulation debugging.
- Implement privacy report explaining:
raw_data_centralized, individual_updates_visible_to_server, client_updates_clipped, dp_noise_added, secure_agg_enabled, risk_level, limitations.
- Add dropout simulation with configurable dropout rate.
- Add non-IID severity setting.

Database tables:

- experiments
- round_metrics

- analytics_results
- privacy_reports
- client_participation

Streamlit dashboard tabs:

1. Overview
2. Federated Analytics
3. Federated Learning Progress
4. Privacy Controls
5. Utility Tradeoff
6. Client Participation
7. Risk Report
8. System Explanation

README must include:

- Problem statement
- Why federated analytics matters
- Why federated learning alone is not automatically private
- Architecture diagram
- Experiment modes
- Privacy mechanisms
- Dataset generation
- Dashboard screenshots
- How to run
- Limitations
- Interview explanation

Do not build a toy script. Build clean, modular, production-style code with tests, comments, and a polished README.

18. Interview Explanation

30-Second Version

"FedPrivacyLab is a federated analytics and federated learning workbench. The idea was to simulate clients that keep raw telemetry data locally. In analytics mode, clients send bounded summaries like counts and latency sums. In learning mode, clients train locally and send model updates. I implemented FedAvg, client selection, dropout simulation, update clipping, secure aggregation as a concept, and DP noise. The dashboard compares privacy strength against utility loss."

Deeper Version

"The main lesson was that federated learning is not automatically private. Raw data stays local, which is good, but individual model updates can still leak information. So the project layers privacy mechanisms: contribution bounding for analytics, update clipping for learning, secure aggregation simulation to hide individual updates, DP noise to reduce leakage, and aggregate-only dashboards."

If asked about secure aggregation

"I implemented secure aggregation as a simulation, not production cryptography. The goal was to model the property that the server should only see the aggregate update, not each individual client update."

If asked about DP

"For analytics, I used Laplace noise on bounded counts and Gaussian noise on clipped sums. For learning, I clipped client model updates and added Gaussian noise to the aggregate update. I also tracked the utility drop through accuracy, ROC-AUC, and relative error."

If asked what makes it graduate-level

"It is not just a FedAvg notebook. It includes non-IID client simulation, client dropout, analytics and learning modes, contribution bounding, update clipping, DP noise, secure aggregation simulation, cohort suppression, privacy reports, utility comparisons, and a dashboard that explains the privacy tradeoffs."

19. Final Build Checklist

- Synthetic client telemetry generation
- Non-IID client profiles
- Federated analytics mode
- Federated learning mode
- FedAvg
- Centralized baseline
- Client selection
- Client dropout simulation
- Contribution bounding
- Update clipping
- Secure aggregation simulation
- DP noise for analytics
- DP noise for model updates
- Privacy report generation
- Utility evaluation
- Streamlit dashboard
- Clean README
- Unit tests

"I built two privacy-preserving systems: one for differentially private telemetry and another for federated analytics/learning. Together, they helped me understand the engineering side of privacy-preserving measurement and ML - exactly the intersection this Apple rotation focuses on."

20. References and Study Links

- Apple Federated Learning with Differential Privacy: <https://machinelearning.apple.com/research/fed-learning-diff-privacy>
- TensorFlow Federated: <https://www.tensorflow.org/federated>
- TensorFlow Federated EMNIST loader: https://www.tensorflow.org/federated/api_docs/python/tff/simulation/datasets/emnist/load_data
- TensorFlow Federated DP tutorial: https://www.tensorflow.org/federated/tutorials/federated_learning_with_differential_privacy
- Flower Framework: <https://flower.ai/docs/framework/index.html>
- Flower Datasets: <https://flower.ai/docs/datasets/>
- LEAF Federated Benchmark: <https://leaf.cmu.edu/>
- Google Practical Secure Aggregation for Federated Learning: <https://research.google/pubs/practical-secure-aggregation-for-federated-learning-on-user-held-data/>